Gauge Equivariant Mesh CNNs
Anisotropic convolutions on geometric graphs

Pim de Haan [*1 2 3] **Maurice Weiler** [*2 3] **Taco Cohen** [1] **Max Welling** [1 3]

Abstract

A common approach to define convolutions on meshes is to interpret them as a graph and apply graph convolutional networks (GCNs). Such GCNs utilize *isotropic* kernels and are therefore insensitive to the relative orientation of vertices and thus to the geometry of the mesh as a whole. We propose Gauge Equivariant Mesh CNNs which generalize GCNs to apply *anisotropic* gauge equivariant kernels. Since the resulting features carry orientation information, we introduce a geometric message passing scheme defined by parallel transporting features over mesh edges. Our experiments validate the significantly improved expressivity of the proposed model over conventional GCNs and other methods.

1. Introduction

Convolutional neural networks (CNNs) have been established as the default method for many machine learning tasks like speech recognition or planar and volumetric image classification and segmentation. Most CNNs are restricted to flat or spherical geometries, where convolutions are easily defined and optimized implementations are available. The empirical success of CNNs on such spaces has generated interest to generalize convolutions to more general spaces like graphs or Riemannian manifolds, creating a field now known as geometric deep learning (Bronstein et al., 2017).

A case of specific interest is convolution on *meshes*, the discrete analog of 2-dimensional embedded Riemannian manifolds. Mesh CNNs can be applied to tasks such as detecting shapes, registering different poses of the same shape and shape segmentation. If we forget the positions of vertices, and which vertices form faces, a mesh M can be represented by a graph $\mathcal{G}$. This allows for the application of *graph convolutional networks* (GCNs) to processing signals on meshes.

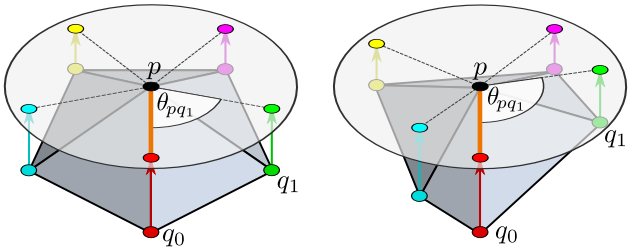


Figure 1. Two local neighbourhoods around vertices p and their representations in the tangent planes T_pM. The distinct geometry of the neighbourhoods is reflected in the different angles θ_{pq_i} of incident edges from neighbours q_i. Graph convolutional networks apply isotropic kernels and can therefore not distinguish both neighbourhoods. Gauge Equivariant Mesh CNNs apply anisotropic kernels and are therefore sensitive to orientations. The arbitrariness of reference orientations, determined by a choice of neighbour q_0, is accounted for by the gauge equivariance of the model.

However, when representing a mesh by a graph, we lose important geometrical information. In particular, in a graph there is no notion of angle between or ordering of two of a node's incident edges (see figure 1). Hence, a GCNs output at a node p is designed to be independent of relative angles and *invariant* to any permutation of its neighbours $q_i \in \mathcal{N}(p)$. A graph convolution on a mesh graph therefore corresponds to applying an *isotropic* convolution kernel. Isotropic filters are insensitive to the orientation of input patterns, so their features are strictly less expressive than those of orientation aware anisotropic filters.

To address this limitation of graph networks we propose *Gauge Equivariant Mesh CNNs* (GEM-CNN), which minimally modify GCNs such that they are able to use anisotropic filters while sharing weights across different positions and respecting the local geometry. One obstacle in sharing anisotropic kernels, which are functions of the angle θ_{pq} of neighbour q with respect to vertex p, over multiple vertices of a mesh is that there is no unique way of selecting a reference neighbour q_0, which has the direction $\theta_{pq_0} = 0$. The reference neighbour, and hence the orientation of the neighbours, needs to be chosen arbitrarily. In order to guar-

[*]Equal contribution [1]Qualcomm AI Research, Amsterdam, NL. Qualcomm AI Research is an initiative of Qualcomm Technologies, Inc. [2]Qualcomm-University of Amsterdam (QUVA) Lab [3]AMLab, University of Amsterdam. Correspondence to: Pim de Haan <pim@qti.qualcomm.com>.

antee the equivalence of the features resulting from different choices of orientations, we adapt Gauge Equivariant CNNs (Cohen et al., 2019b) to general meshes. The kernels of our model are thus designed to be *equivariant under gauge transformations*, that is, to guarantee that the responses for different kernel orientations are related by a prespecified transformation law. Such features are identified as geometric objects like scalars, vectors, tensors, etc., depending on the specific choice of transformation law. In order to compare such geometric features at neighbouring vertices, they need to be *parallel transported* along the connecting edge.

In our implementation we first specify the transformation laws of the feature spaces and compute a space of gauge equivariant kernels. Then we pick arbitrary reference orientations at each node, relative to which we compute neighbour orientations and compute the corresponding edge transporters. Given these quantities, we define the forward pass as a message passing step via edge transporters followed by a contraction with the equivariant kernels evaluated at the neighbour orientations. Algorithmically, Gauge Equivariant Mesh CNNs are therefore just GCNs with anisotropic, gauge equivariant kernels and message passing via parallel transporters. Conventional GCNs are covered in this framework for the specific choice of isotropic kernels and trivial edge transporters, given by identity maps.

In Sec. 2, we will give an outline of our method, deferring details to Secs. 3, 4 and 5. In our experiments in Sec. 7, we find that the enhanced expressiveness of Gauge Equivariant Mesh CNNs enables them to outperform conventional GCNs and other prior work in a shape correspondence task.

2. Convolutions on Graphs with Geometry

We consider the problem of processing signals on discrete 2-dimensional manifolds, or meshes M. Such meshes are described by a set $\mathcal{V}$ of vertices in $\mathbb{R}^3$ together with a set $\mathcal{F}$ of tuples, each consisting of the vertices at the corners of a face. For a mesh to describe a proper manifold, each edge needs to be connected to two faces, and the neighbourhood of each vertex needs to be homeomorphic to a disk. Mesh M induces a graph $\mathcal{G}$ by forgetting the coordinates of the vertices while preserving the edges.

A conventional graph convolution between kernel K and signal f, evaluated at a vertex p, can be defined by

$$(K \star f)_p \;=\; K_{\text{self}} f_p \;+\; \sum_{q \in \mathcal{N}_p} K_{\text{neigh}} f_q, \qquad (1)$$

where $\mathcal{N}_p$ is the set of neighbours of p in $\mathcal{G}$, and $K_{\text{self}} \in \mathbb{R}^{C_{\text{in}} \times C_{\text{out}}}$ and $K_{\text{neigh}} \in \mathbb{R}^{C_{\text{in}} \times C_{\text{out}}}$ are two linear maps which model a self interaction and the neighbour contribution, respectively. Importantly, graph convolution does not distinguish different neighbours, because each feature vector f_q is multiplied by the same matrix K_{neigh} and then summed. For this reason we say the kernel is *isotropic*.

Algorithm 1 Gauge Equivariant Mesh CNN layer

Input: mesh M, input/output feature types $\rho_{\text{in}}, \rho_{\text{out}}$, reference neighbours $(q_0^p \in \mathcal{N}_p)_{p \in M}$.

Compute basis kernels $K^i_{\text{self}}, K^i_{\text{neigh}}(\theta)$ ▷ Sec. 3

Initialise weights w^i_{self} and w^i_{neigh}.

For each neighbour pair, $p \in M, q \in \mathcal{N}_p$: ▷ Sec. 4.
compute neighbor angles θ_{pq} relative to reference neighbor
compute parallel transporters $g_{q \to p}$

Forward$\big($input features $(f_p)_{p \in M}$, weights $w^i_{\text{self}}, w^i_{\text{neigh}}\big)$:

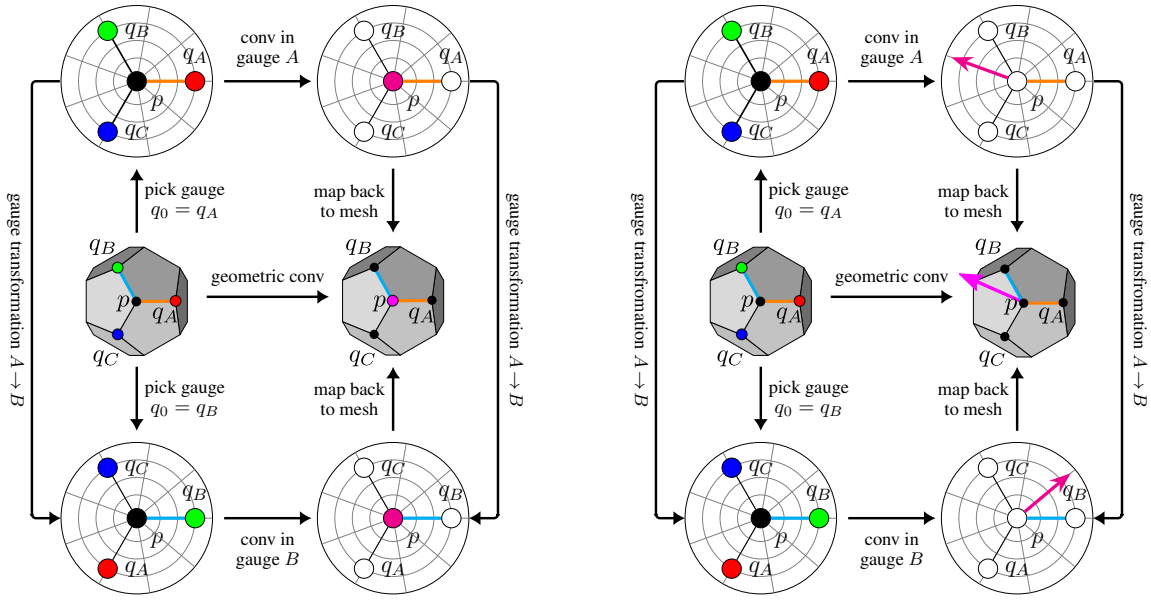
$$f'_p \leftarrow \sum_i w^i_{\text{self}} K^i_{\text{self}} f_p + \sum_{i, q \in \mathcal{N}_p} w^i_{\text{neigh}} K^i_{\text{neigh}}(\theta_{pq}) \rho_{\text{in}}(g_{q \to p}) f_q$$

Consider the example in figure 1, where on the left and right, the neighbourhood of one vertex p, containing neighbours $q \in \mathcal{N}_p$, is visualized. An isotropic kernel would propagate the signal from the neighbours to p in exactly the same way in both neighbourhoods, even though the neighbourhoods are geometrically distinct. For this reason, our method uses direction sensitive (*anisotropic*) kernels instead of isotropic kernels. Anisotropic kernels are inherently more expressive than isotropic ones which is why they are used universally in conventional planar CNNs.

We propose the Gauge Equivariant Mesh Convolution, a minimal modification of graph convolution that allows for anisotropic kernels $K(\theta)$ whose value depends on an orientation $\theta \in [0, 2\pi)$. To define the orientations θ_{pq} of neighbouring vertices $q \in \mathcal{N}_p$ of p, we first map them to the tangent plane T_pM at p, as visualized in figure 1. We then pick an *arbitrary* reference neighbour q_0^p to determine a reference orientation[1] $\theta_{pq_0^p} := 0$, marked orange in figure 1. This induces a basis on the tangent plane, which, when expressed in polar coordinates, defines the angles θ_{pq} of the other neighbours.

As we will motivate in the next section, features in a Gauge Equivariant CNN are coefficients of geometric quantities. For example, a tangent vector at vertex p can be described either geometrically by a 3 dimensional vector orthogonal to the normal at p or by two coefficients in the basis on the tangent plane. In order to perform convolution, geometric features at different vertices need to be linearly combined, for which it is required to first "parallel transport" the features to the same vertex. This is done by applying a matrix $\rho(g_{q \to p}) \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}$ to the coefficients of the feature at q, in order to obtain the coefficients of the feature vector transported to p, which can be used for the convolution at p. The transporter depends on the geometric *type* of the feature, denoted by ρ. Details of how the tangent space is defined,

[1] Mathematically, this corresponds to a choice of *local reference frame* or *gauge*.



(a) Convolution from scalar to scalar features.

(b) Convolution from scalar to vector features.

Figure 2. Visualization of the Gauge Equivariant Mesh Convolution in two configurations, scalar to scalar and scalar to vector. The convolution operates in a gauge, so that vectors are expressed in coefficients in a basis and neighbours have polar coordinates, but can also be seen as a *geometric convolution*, a gauge-independent map from an input signal on the mesh to a output signal on the mesh. The convolution is equivariant if this geometric convolution does not depend on the intermediate chosen gauge, so if the diagram commutes.

how to compute the map to the tangent space, angles θ_{pq}, and the parallel transporter are given in Sec 4.

In combination, this leads to the GEM-CNN convolution

$$(K \star f)_p = K_{\text{self}} f_p + \sum_{q \in \mathcal{N}_p} K_{\text{neigh}}(\theta_{pq}) \rho(g_{q \to p}) f_q \qquad (2)$$

which differs from the conventional graph convolution, defined in Eq. 1 only by the use of an anisotropic kernel and the parallel transport message passing.

We require the outcome of the convolution to be *equivalent* for any choice of reference orientation. This is not the case for any anisotropic kernel but only for those which are *equivariant under changes of reference orientations* (gauge transformations). Equivariance imposes a linear constraint on the kernels. We therefore solve for complete sets of "basis-kernels" K^i_{self} and K^i_{neigh} satisfying this constraint and linearly combine them with parameters w^i_{self} and w^i_{neigh} such that $K_{\text{self}} = \sum_i w^i_{\text{self}} K^i_{\text{self}}$ and $K_{\text{neigh}} = \sum_i w^i_{\text{neigh}} K^i_{\text{neigh}}$. Details on the computation of basis kernels are given in section 3. The full algorithm for initialisation and forward pass, which is of time and space complexity linear in the number of vertices, for a GEM-CNN layer are listed in algorithm 1. Gradients can be computed by automatic differentiation.

3. Gauge Equivariance & Geometric Features

On a general mesh, the choice of the reference neighbour, or gauge, which defines the orientation of the kernel, can only be made arbitrarily. However, this choice should not arbitrarily affect the outcome of the convolution, as this would impede the generalization between different locations and different meshes. Instead, Gauge Equivariant Mesh CNNs have the property that their output transforms according to a known rule as the gauge changes.

Consider the left hand side of figure 2(a). Given a neighbourhood of vertex p, we want to express each neighbour q in terms of its polar coordinates (r_q, θ_q) on the tangent plane, so that the kernel value at that neighbour $K_{\text{neigh}}(\theta_q)$ is well defined. This requires choosing a basis on the tangent plane, determined by picking a neighbour as reference neighbour (denoted q_0), which has the zero angle $\theta_{q_0} = 0$. In the top path, we pick q_A as reference neighbour. Let us call this gauge A, in which neighbours have angles θ^A_q. In the bottom path, we instead pick neighbour q_B as reference point and are in gauge B. We get a different basis for the tangent plane and different angles θ^B_q for each neighbour. Comparing the two gauges, we see that they are related by a rotation, so that $\theta^B_q = \theta^A_q - \theta^A_{q_B}$. This change of gauge is called a gauge transformation of angle $g := \theta^A_{q_B}$.

In figure 2(a), we illustrate a gauge equivariant convolution that takes input and output features such as gray scale image

values on the mesh, which are called scalar features. The top path represents the convolution in gauge A, the bottom path in gauge B. In either case, the convolution can be interpreted as consisting of three steps. First, for each vertex p, the value of the scalar features on the mesh at each neighbouring vertex q, represented by colors, is mapped to the tangent plane at p at angle θ_q defined by the gauge. Subsequently, the convolutional kernel sums for each neighbour q, the product of the feature at q and kernel $K(\theta_q)$. Finally the output is mapped back to the mesh. These three steps can be composed into a single step, which we could call a *geometric convolution*, mapping from input features on the mesh to output features on the mesh. The convolution is *gauge equivariant* if this geometric convolution does not depend on the gauge we pick in the interim, so in figure 2(a), if the convolution in the top path in gauge A has same result the convolution in the bottom path in gauge B, making the diagram commute. In this case, however, we see that the convolution output needs to be the same in both gauges, for the convolution to be equivariant. Hence, we must have that $K(\theta_q) = K(\theta_q - g)$, as the orientations of the neighbours differ by some angle g, and the kernel must be isotropic.

As we aim to design an anisotropic convolution, the output feature of the convolution at p can, instead of a scalar, be two numbers $v \in \mathbb{R}^2$, which can be interpreted as coefficients of a tangent feature vector in the tangent space at p, visualized in figure 2(b). As shown on the right hand side, different gauges induce a different basis of the tangent plane, so that the *same tangent vector* (shown on the middle right on the mesh), is represented by *different coefficients* in the gauge (shown on the top and bottom on the right). This gauge equivariant convolution must be anisotropic: going from the top row to the bottom row, if we change orientations of the neighbours by $-g$, the coefficients of the output vector $v \in \mathbb{R}^2$ of the kernel must be also rotated by $-g$. This is written as $R(-g)v$, where $R(-g) \in \mathbb{R}^{2\times 2}$ is the matrix that rotates by angle $-g$.

Vectors and scalars are not the only kind of geometric features that can be inputs and outputs of a GEM-CNN layer. In general, the coefficients of a geometric feature of C dimensions changes by a linear transformation $\rho(-g) \in \mathbb{R}^{C\times C}$ if the gauge is rotated by angle g. The map $\rho : [0, 2\pi) \to \mathbb{R}^{C\times C}$ is called the *type* of the geometric quantity and is formally known as a group representation of the planar rotation group $\mathrm{SO}(2)$. From the theory of group representations, we know that any feature type can be composed from "irreducible representations" (irreps). For $\mathrm{SO}(2)$, these are the one dimensional invariant scalar representation ρ_0 and for all $n \in \mathbb{N}_{>0}$, a two dimensional representation ρ_n,

$$\rho_0(g) = 1, \quad \rho_n(g) = \begin{pmatrix} \cos ng & -\sin ng \\ \sin ng & \cos ng \end{pmatrix}.$$

$\rho_{\text{in}} \to \rho_{\text{out}}$	linearly independent solutions for $K_{\text{neigh}}(\theta)$
$\rho_0 \to \rho_0$	(1)
$\rho_n \to \rho_0$	$(\cos n\theta \ \ \sin n\theta)\,, (\sin n\theta \ \ -\cos n\theta)$
$\rho_0 \to \rho_m$	$\begin{pmatrix}\cos m\theta \\ \sin m\theta\end{pmatrix}, \begin{pmatrix}\sin m\theta \\ -\cos m\theta\end{pmatrix}$
$\rho_n \to \rho_m$	$\begin{pmatrix}c_- & -s_- \\ s_- & c_-\end{pmatrix}, \begin{pmatrix}s_- & c_- \\ -c_- & s_-\end{pmatrix}, \begin{pmatrix}c_+ & s_+ \\ s_+ & -c_+\end{pmatrix}, \begin{pmatrix}-s_+ & c_+ \\ c_+ & s_+\end{pmatrix}$
$\rho_{\text{in}} \to \rho_{\text{out}}$	linearly independent solutions for K_{self}
$\rho_0 \to \rho_0$	(1)
$\rho_n \to \rho_n$	$\begin{pmatrix}1 & 0 \\ 0 & 1\end{pmatrix}, \begin{pmatrix}0 & 1 \\ -1 & 0\end{pmatrix}$

Table 1. Solutions to the angular kernel constraint for kernels that map from ρ_n to ρ_m. We denote $c_\pm = \cos((m \pm n)\theta)$ and $s_\pm = \sin((m \pm n)\theta)$.

Scalars and tangent vector features correspond to ρ_0 and ρ_1 respectively and we have $R(g) = \rho_1(g)$.

The type of the feature at each layer in the network can thus be fully specified (up to a change of basis) by the number of copies of each irrep. Similar to the dimensionality in a conventional CNN, the choice of type is a hyperparameter that can be freely chosen to optimize performance.

We use a notation, such that, for example, $\rho = 1\rho_0 \oplus 2\rho_1$ means that the feature contains one ρ_0 irrep, which is a scalar, and two ρ_1 irreps, which are vectors. This feature is five dimensional ($1\times\dim\rho_0 + 2\times\dim\rho_1 = 1\times 1 + 2\times 2 = 5$) and transforms transforms with block diagonal matrix:

$$\rho(g) = \begin{pmatrix} 1 & & & & \\ & \cos g & -\sin g & & \\ & \sin g & \cos g & & \\ & & & \cos g & -\sin g \\ & & & \sin g & \cos g \end{pmatrix}$$

3.1. Kernel Constraint

Given an input type ρ_{in} and output type ρ_{out} of dimensions C_{in} and C_{out}, the kernels are $K_{\text{self}} \in \mathbb{R}^{C_{\text{out}}\times C_{\text{in}}}$ and $K_{\text{neigh}} : [0, 2\pi) \to \mathbb{R}^{C_{\text{out}}\times C_{\text{in}}}$. However, not all such kernels are equivariant. Consider again examples figure 2(a) and figure 2(b). If we map from a scalar to a scalar, we get that $K_{\text{neigh}}(\theta - g) = K_{\text{neigh}}(\theta)$ for all angles θ, g and the convolution is isotropic. If we map from a scalar to a vector, we get that rotating the angles θ_q results in the same tangent vector as rotating the output vector coefficients, so that $K_{\text{neigh}}(\theta - g) = R(-g)K_{\text{neigh}}(\theta)$.

In general, as derived by Cohen et al. (2019b) and in appendix A, the kernels must satisfy for any gauge transformation $g \in [0, 2\pi)$ and angle $\theta \in [0, 2\pi)$, that

$$K_{\text{neigh}}(\theta - g) = \rho_{\text{out}}(-g)\ K_{\text{neigh}}(\theta)\ \rho_{\text{in}}(g), \tag{3}$$

$$K_{\text{self}} = \rho_{\text{out}}(-g)\ K_{\text{self}}\ \rho_{\text{in}}(g). \tag{4}$$

The kernel can be seen as consisting of multiple blocks,

where each block takes as input one irrep and outputs one irrep. For example if ρ_{in} would be of type $1\rho_0 \oplus 2\rho_1$ and ρ_{out} of type $1\rho_1 \oplus 1\rho_3$, we have

$$K_{\text{neigh}}(\theta) = \begin{pmatrix} K_{10}(\theta) & K_{11}(\theta) & K_{11}(\theta) \\ K_{30}(\theta) & K_{31}(\theta) & K_{31}(\theta) \end{pmatrix} \in \mathbb{R}^{4\times 5}.$$

where e.g. $K_{31}(\theta) \in \mathbb{R}^{2\times 2}$ is a kernel that takes as input irrep ρ_1 and as output irrep ρ_3 and needs to satisfy Eq. 3. As derived by Weiler & Cesa (2019) and in Appendix B, the kernels $K_{\text{neigh}}(\theta)$ and K_{self} mapping from irrep ρ_n to irrep ρ_m can be written as a linear combination of the basis kernels listed in table 1. The table shows that equivariance requires the self-interaction to only map from one irrep to the same irrep. Hence, we have $K_{\text{self}} = \begin{pmatrix} 0 & K_{11} & K_{11} \\ 0 & 0 & 0 \end{pmatrix} \in \mathbb{R}^{4\times 3}$.

All basis-kernels of all pairs of input irreps and output irreps can be linearly combined to form an arbitrary equivariant kernel from feature of type ρ_{in} to ρ_{out}. In the above example, we have $2 \times 2 + 4 \times 4 = 20$ basis kernels for K_{neigh} and 4 basis kernels for K_{self}. The layer thus has 24 parameters.

4. Geometry & Parallel Transport

A gauge, or choice of reference neighbor at each vertex, fully determines the neighbor orientations θ_{pq} and the parallel transporters $g_{q\to p}$ along edges. The following two subsections give details on how to compute these quantities.

4.1. Local neighborhood geometry

Neighbours q of vertex p can be mapped uniquely to the tangent plane at p using a map called the Riemannnian logarithmic map, visualized in figure 1. A choice of reference neighbor then determines a reference frame in the tangent space which assigns polar coordinates to all other neighbors. The neighbour orientations θ_{pq} are the angular components of each neighbor in this polar coordinate system.

We define the tangent space T_pM at vertex p as that two dimensional subspace of $\mathbb{R}^3$, which is determined by a normal vector n given by the area weighted average of the normal vectors of the adjacent mesh faces. While the tangent spaces are two dimensional, we implement them as being embedded in the ambient space $\mathbb{R}^3$ and therefore represent their elements as three dimensional vectors. The reference frame corresponding to the chosen gauge, defined below, allows to identify these 3-vectors by their coefficient 2-vectors.

Each neighbor q is represented in the tangent space by the vector $\log_p(q) \in T_pM$ which is computed via the discrete analog of the Riemannian logarithm map. We define this map $\log_p : \mathcal{N}_p \to T_pM$ for neighbouring nodes as the projection of the edge vector $q - p$ on the tangent plane, followed by a rescaling such that the norm $|\log_p(q)| = |q - p|$ is preserved. Writing the projection operator on the tangent plane as $(\mathbb{1} - nn^\top)$, the logarithmic map is thus given by:

$$\log_p(q) \;:=\; |q-p| \frac{(\mathbb{1} - nn^\top)(q-p)}{|(\mathbb{1} - nn^\top)(q-p)|}$$

Geometrically, this map can be seen as "folding" each edge up to the tangent plane, and therefore encodes the orientation of edges and preserves their lengths.

The normalized reference edge vector $\log_p(q_0)$ uniquely determines a right handed, orthonormal reference frame $(e_{p,1},\, e_{p,2})$ of T_pM by setting $e_{p,1} := \log_p(q_0)/|\log_p(q_0)|$ and $e_{p,2} := n \times e_{p,1}$. The angle θ_{pq} is then defined as the angle of $\log_p(q)$ in polar coordinates corresponding to this reference frame. Numerically, it can be computed by

$$\theta_{pq} \;:=\; \operatorname{atan2}\left(e_{p,2}^\top \log_p(q),\; e_{p,1}^\top \log_p(q)\right).$$

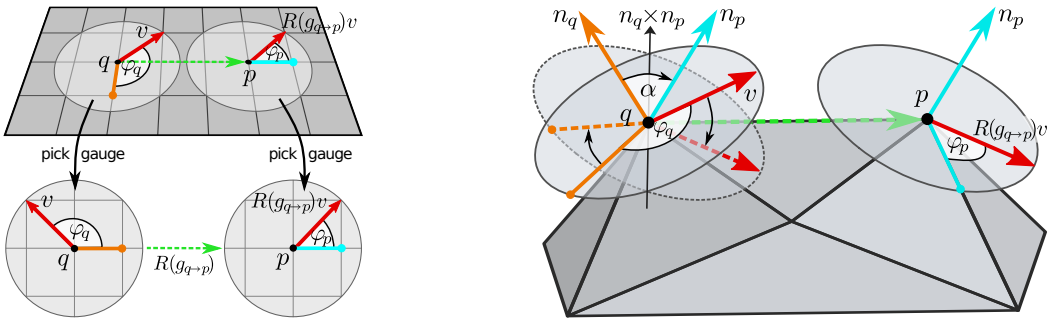
Given the reference frame $(e_{p,1}, e_{p,2})$, a 2-tuple of coefficients $(v_1, v_2) \in \mathbb{R}^2$ specifies an (embedded) tangent vector $v_1 e_{p,1} + v_2 e_{p,2} \in T_pM \subset \mathbb{R}^3$. This assignment is formally given by the *gauge map* $E_p : \mathbb{R}^2 \to T_pM \subset \mathbb{R}^3$ which is a vector space isomorphism. In our case, it can be identified with the matrix

$$E_p = \begin{bmatrix} | & | \\ e_{p,1} & e_{p,2} \\ | & | \end{bmatrix} \in \mathbb{R}^{3\times 2}. \tag{5}$$

4.2. Parallel edge transporters

On curved meshes, feature vectors f_q and f_p at different locations q and p are expressed in different gauges, which makes it geometrically invalid to accumulate their information directly. Instead, when computing a new feature at p, the neighboring feature vectors at $q \in \mathcal{N}_p$ first have to be parallel transported into the feature space at p before they can be processed. The parallel transport along the edges of a mesh is determined by the (discrete) Levi-Civita connection corresponding to the metric induced by the ambient space $\mathbb{R}^3$. This connection is given by parallel transporters $g_{q\to p} \in [0, 2\pi)$ on the mesh edges which map tangent vectors $v_q \in T_qM$ at q to tangent vectors $R(g_{q\to p})v_q \in T_pM$ at p. Feature vectors f_q of type ρ are similarly transported to $\rho(g_{q\to p})f_q$ by applying the corresponding feature vector transporter $\rho(g_{q\to p})$.

In order to build some intuition, it is illustrative to first consider transporters on a planar mesh. In this case the parallel transport can be thought of as moving a vector along an edge without rotating it. The resulting abstract vector is then parallel to the original vector in the usual sense on flat spaces, see figure 3(a). However, if the (transported) source frame at q disagrees with the target frame at p, the *coefficients* of the transported vector have to be transformed to the target coordinates. This coordinate transformation



(a) Parallel transport on a flat mesh.

(b) Parallel transport along an edge of a general mesh.

Figure 3. Parallel transport of tangent vectors $v \in T_qM$ at q to $R(g_{q\to p})v \in T_pM$ at p on meshes. On a flat mesh, visualized in figure 3(a), parallel transport moves a vector such that it stays parallel in the usual sense on flat spaces. The parallel transporter $g_{q\to p} = \varphi_p - \varphi_q$ corrects the transported vector *coefficients* for differing gauges at q and p. When transporting along the edge of a general mesh, the tangent spaces at q and p might not be aligned, see figure 3(b). Before correcting for the relative frame orientation via $g_{q\to p}$, the tangent space T_qM, and thus $v \in T_qM$, is rotated by an angle α around $n_q \times n_p$ such that its normal n_q coincides with that of n_p.

from polar angles φ_q of v to φ_p of $R(g_{q\to p})v$ defines the transporter $g_{q\to p} = \varphi_p - \varphi_q$.

On general meshes one additionally has to account for the fact that the tangent spaces $T_qM \subset \mathbb{R}^3$ and $T_pM \subset \mathbb{R}^3$ are usually not parallel in the ambient space $\mathbb{R}^3$. The parallel transport therefore includes the additional step of first aligning the tangent space at q to be parallel to that at p, before translating a vector between them, see figure 3(b). In particular, given the normals n_q and n_p of the source and target tangent spaces T_qM and T_pM, the source space is being aligned by rotating it via $R_\alpha \in \mathrm{SO}(3)$ by an angle $\alpha = \arccos(n_q^\top n_p)$ around the axis $n_q \times n_p$ in the ambient space. Denote the rotated source frame by $(R_\alpha e_{q,1}, R_\alpha e_{q,2})$ and the target frame by $(e_{p,1}, e_{p,2})$. The angle to account for the parallel transport between the two frames, defining the discrete Levi-Civita connection on mesh edges, is then found by computing

$$g_{q\to p} \;=\; \mathrm{atan2}\left((R_\alpha e_{q,2})^\top e_{p,1},\; (R_\alpha e_{q,1})^\top e_{p,1}\right). \qquad (6)$$

In practice we precompute these connections before training a model.

Under gauge transformations by angles g_p at p and g_q at q the parallel transporters transform according to

$$g_{q\to p} \;\mapsto\; g_p + g_{q\to p} - g_q\,. \qquad (7)$$

Intuitively, this transformation states that a transporter in a transformed gauge is given by a gauge transformation back to the original gauge via $-g_q$ followed by the original transport by $g_{q\to p}$ and a transformation back to the new gauge via g_p.

For more details on discrete connections and transporters, extending to arbitrary paths e.g. over faces, we refer to (Lai et al., 2009; Crane et al., 2010; 2013).

5. Non-linearity

Besides convolutional layers, the GEM-CNN contains non-linear layers, which also need to be gauge equivariant, for the entire network to be gauge equivariant. The coefficients of features built out of irreducible representaions, as described in section 3, do not commute with point-wise non-linearities (Worrall et al., 2017; Thomas et al., 2018; Weiler et al., 2018a; Kondor et al., 2018). Norm non-linearities and gated non-linearities (Weiler & Cesa, 2019) can be used with such features, but generally perform worse in practice compared to point-wise non-linearities (Weiler & Cesa, 2019). Hence, we propose the *RegularNonlinearity*, which uses point-wise non-linearities and is approximately gauge equivariant.

This non-linearity is built on Fourier transformations. Consider a continuous periodic signal, on which we perform a band-limited Fourier transform with band limit b, obtaining $2b+1$ Fourier coefficients. If this continuous signal is shifted by an arbitrary angle g, then the corresponding Fourier components transform with linear transformation $\rho_{0:b}(-g)$, for $2b+1$ dimensional representation $\rho_{0:b} := \rho_0 \oplus \rho_1 \oplus \ldots \oplus \rho_b$.

It would be exactly equivariant to take a feature of type $\rho_{0:b}$, take a continuous inverse Fourier transform to a continuous periodic signal, then apply a point-wise non-linearity to that signal, and take the continuous Fourier transform, to recover a feature of type $\rho_{0:b}$. However, for implementation, we use N intermediate samples and the discrete Fourier transform. This is exactly gauge equivariant for gauge transformation of angles multiple of $2\pi/N$, but only approximately equivariant for other angles. It can be shown that as $N \to \infty$, the non-linearity is exactly gauge equivariant.

6. Related Work

The irregular structure of meshes leads to a variety of approaches to define convolutions. Closely related to our method are graph based methods which are often based on variations of graph convolutional networks (Kipf & Welling, 2017; Defferrard et al., 2016). GCNs have been applied on spherical meshes (Perraudin et al., 2019) and cortical surfaces (Cucurull et al., 2018; Zhao et al., 2019). Verma et al. (2018) augment GCNs with anisotropic kernels which are dynamically computed via an attention mechanism over graph neighbours.

Instead of operating on the graph underlying a mesh, several approaches leverage its geometry by treating it as a discrete manifold. Convolution kernels can then be defined in geodesic polar coordinates which corresponds to a projection of kernels from the tangent space to the mesh via the exponential map. This allows for kernels that are larger than the immediate graph neighbourhood and message passing over faces but does not resolve the issue of ambiguous kernel orientation. Masci et al. (2015); Monti et al. (2016) and Sun et al. (2018) address this issue by restricting the network to orientation invariant features which are computed by applying anisotropic kernels in several orientations and pooling over the resulting responses. The models proposed in (Boscaini et al., 2016) and (Schonsheck et al., 2018) are explicitly gauge dependent with preferred orientations chosen via the principal curvature direction and the parallel transport of kernels, respectively. Poulenard & Ovsjanikov (2018) proposed a non-trivially gauge equivariant network based on geodesic convolutions, however, the model parallel transports only partial information of the feature vectors, corresponding to certain kernel orientations.

Another class of approaches defines spectral convolutions on meshes However, as argued in (Bronstein et al., 2017), the Fourier spectrum of a mesh depends heavily on its geometry, which makes such methods instable under deformations and impedes the generalization between different meshes. Spectral convolutions further correspond to isotropic kernels.

A construction based on toric covering maps of topologically spherical meshes was presented in (Maron et al., 2017). An entirely different approach to mesh convolutions is to apply a linear map to a spiral of neighbours (Bouritsas et al., 2019; Gong et al., 2019), which works well only for meshes with a similar graph structure.

On flat Euclidean spaces our method corresponds to Steerable CNNs (Cohen & Welling, 2017; Weiler et al., 2018a; Weiler & Cesa, 2019; Cohen et al., 2019a). As our model, these networks process geometric feature fields of types ρ and are equivariant under gauge transformations, however, due to the flat geometry, the parallel transporters become trivial. Regular nonlinearities are on flat spaces used in group convolutional networks (Cohen & Welling, 2016; Weiler et al., 2018b; Hoogeboom et al., 2018; Bekkers et al., 2018; Winkels & Cohen, 2018; Worrall & Brostow, 2018; Worrall & Welling, 2019; Sosnovik et al., 2020).

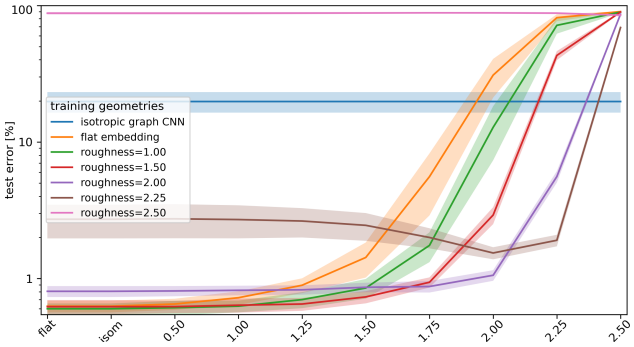


Figure 4. Test errors for MNIST digit classification on embedded meshes. Except for the isotropic graph CNN, different curves correspond to the same GEM-CNN model, trained on different training geometries. The x-axis shows different test geometries on which each model is tested. Shaded regions state the standard errors of the means over 6 runs.

7. Experiments

We examine the performance of the GEM-CNN and the influence of varying geometry in two experiments.

7.1. Embedded MNIST

We first investigate how Gauge Equivariant Mesh CNNs perform on, and generalize between, different mesh geometries. For this purpose we conduct simple MNIST digit classification experiments on embedded rectangular meshes of 28×28 vertices. As a baseline geometry we consider a flat mesh as visualized in figure 5(a). A second type of geometry is defined as different *isometric* embeddings of the flat mesh, see figure 5(b). Note that this implies that the *intrinsic* geometry of these isometrically embedded meshes is indistinguishable from that of the flat mesh. To generate geometries which are intrinsically curved, we add random normal displacements to the flat mesh. We control the amount of curvature by smoothing the resulting displacement fields with Gaussian kernels of different widths σ and define the roughness of the resulting mesh as $3 - \sigma$. Figures 5(c)-5(h) show the results for roughnesses of 0.5, 1, 1.5, 2, 2.25 and 2.5. For each of the considered settings we generate 32 different train and 32 test geometries.

To test the performance on, and generalization between, different geometries, we train equivalent GEM-CNN models on a flat mesh and meshes with a roughness of 1, 1.5, 2, 2.25 and 2.5. Each model is tested individually on each of the considered test geometries, which are the flat mesh, isometric embeddings and curved embeddings with a roughness of 0.5, 1, 1.25, 1.5, 1.75, 2, 2.25 and 2.5. Figure 4

shows the test errors of the GEM-CNNs on the different train geometries (different curves) for all test geometries (shown on the x-axis). Since our model is purely defined in terms of the intrinsic geometry of a mesh, it is expected to be insensitive to isometric changes in the embeddings. This is empirically confirmed by the fact that the test performances on flat and isometric embeddings are exactly equal. As expected, the test error increases for most models with the surface roughness. Models trained on more rough surfaces are hereby more robust to deformations. The models generalize well from a rough training to smooth test geometry up to a training roughness of 1.5. Beyond that point, the test performances on smooth meshes degrades up to the point of random guessing at a training roughness of 2.5.

As a baseline, we build an *isotropic* graph CNN with the same network topology and number of parameters ($\approx 163k$). This model is insensitive to the mesh geometry and therefore performs exactly equal on all surfaces. While this enhances its robustness on very rough meshes, its test error of $19.80 \pm 3.43\%$ is an extremely bad result on MNIST. In contrast, the use of anisotropic filters of GEM-CNN allows it to reach a test error of only $0.60 \pm 0.05\%$ on the flat geometry. It is therefore competitive with conventional CNNs on pixel grids, which apply anisotropic kernels as well. More details on the datasets, models and further experimental setup are given in appendix C.1.

7.2. Shape Correspondence

As a second experiment, we perform non-rigid shape correspondence on the FAUST dataset (Bogo et al., 2014), following Masci et al. (2015) [2] . The data consists of 100 meshes of human bodies in various positions, split into 80 meshes for training and 20 for testing. The vertices are registered, such that vertices on the same position on the body, such as the tip of the left thumb, have the same identifier on all meshes. All meshes have 6890 vertices, making this a 6890-class segmentation problem.

We use a simple architecture, which transforms the XYZ coordinates of each vertex, which is of type $3\rho_0$, using 6 convolutional layers to a feature of type $64\rho_0$, with intermediate features of type $16\rho_0 \oplus 16\rho_1 \oplus 16\rho_2$. The convolutional layers use residual connections and the RegularNonlinearity with $N = 5$ samples. Afterwards, we use two 1×1 convolutions with ReLU to map first to 256 channels and finally to 6890 channels, after which a softmax predicts the registration probabilities. The 1×1 convolutions use a dropout of 50% and 1E-4 weight decay. The network is trained with a cross entropy loss with an initial learning rate of 0.01, which is halved when training loss reaches a plateau.

As all figures in the FAUST data set are similarly meshed and oriented, breaking the gauge equivariance in higher layers can actually be beneficial. As shown in (Weiler & Cesa, 2019), symmetry can be broken by treating non-invariant features as invariant features as input to the final 1×1 convolution. Such architectures are equivariant on lower levels, while allowing orientation sensitivity at higher layers.

[2]These experiments were executed on QUVA machines.

Model	Features	Accuracy
ACNN (Boscaini et al., 2016)	SHOT	62.4 %
Geodesic CNN (Masci et al., 2015)	SHOT	65.4 %
MoNet (Monti et al., 2016)	SHOT	73.8 %
FeaStNet (Verma et al., 2018)	XYZ	98.7%
ZerNet (Sun et al., 2018)	XYZ	96.9%
SpiralNet++ (Gong et al., 2019)	XYZ	99.8%
Graph CNN	XYZ	1.40 ± 0.5 %
Graph CNN	SHOT	23.80 ± 8%
GEM-CNN	XYZ	99.73 ± 0.04 %
GEM-CNN (broken symmetry)	XYZ	**99.89** ± 0.02 %

Table 2. Results of FAUST shape correspondence. Statistics are means and standard errors of the mean of over three runs. All cited results are from their respective papers.

As baselines, we compare to various models, some of which use more complicated pipelines, such as (1) the computation of geodesics over the mesh, which requires solving partial differential equations, (2) pooling, which requires finding a uniform sub-selection of vertices, (3) the pre-computation of SHOT features which locally describe the geometry (Salti et al., 2014), or (4) post-processing refinement of the predictions. The GEM-CNN requires none of these additional steps. In addition, we compare to SpiralNet++ (Gong et al., 2019), which requires all inputs to be similarly meshed. Finally, we compare to an isotropic version of the GEM-CNN, which reduces to a conventional graph CNN. The results in table 2 show that the GEM-CNN outperforms prior works, that isotropic graph CNNs are unable to solve the task and that for this data set, breaking gauge symmetry in the final layers of the network is beneficial.

8. Conclusions

Convolutions on meshes are commonly performed as a convolution on their underlying graph, by forgetting geometry, such as orientation of neighbouring vertices. In this paper we propose Gauge Equivariant Mesh CNNs, which endow Graph Convolutional Networks on meshes with anisotropic kernels and parallel transport. Hence, they are sensitive to the mesh geometry, and result in equivalent outputs regardless of the arbitrary choice of kernel orientation.

We demonstrate that the inference of GEM-CNNs is invariant under isometric deformations of meshes and generalizes well over a range of non-isometric deformations. On the FAUST shape correspondence task, we show that Gauge equivariance, combined with symmetry breaking in the final layer, leads to state of the art performance.

References

Bekkers, E. J., Lafarge, M. W., Veta, M., Eppenhof, K. A., Pluim, J. P., and Duits, R. Roto-translation covariant convolutional networks for medical image analysis. In *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2018.

Bogo, F., Romero, J., Loper, M., and Black, M. J. Faust: Dataset and evaluation for 3d mesh registration. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3794–3801, 2014.

Boscaini, D., Masci, J., Rodolà, E., and Bronstein, M. M. Learning shape correspondence with anisotropic convolutional neural networks. In *NIPS*, 2016.

Bouritsas, G., Bokhnyak, S., Ploumpis, S., Bronstein, M., and Zafeiriou, S. Neural 3d morphable models: Spiral convolutional networks for 3d shape representation learning and generation. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 7213–7222, 2019.

Bronstein, M. M., Bruna, J., LeCun, Y., Szlam, A., and Vandergheynst, P. Geometric deep learning: Going beyond Euclidean data. *IEEE Signal Processing Magazine*, 2017.

Cohen, T. and Welling, M. Group equivariant convolutional networks. In *ICML*, 2016.

Cohen, T. S. and Welling, M. Steerable CNNs. In *ICLR*, 2017.

Cohen, T. S., Geiger, M., and Weiler, M. A general theory of equivariant CNNs on homogeneous spaces. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2019a.

Cohen, T. S., Weiler, M., Kicanaoglu, B., and Welling, M. Gauge equivariant convolutional networks and the Icosahedral CNN. 2019b.

Crane, K., Desbrun, M., and Schröder, P. Trivial connections on discrete surfaces. *Computer Graphics Forum (SGP)*, 29(5):1525–1533, 2010.

Crane, K., de Goes, F., Desbrun, M., and Schröder, P. Digital geometry processing with discrete exterior calculus. In *ACM SIGGRAPH 2013 courses*, SIGGRAPH '13, New York, NY, USA, 2013. ACM.

Cucurull, G., Wagstyl, K., Casanova, A., Veličković, P., Jakobsen, E., Drozdzal, M., Romero, A., Evans, A., and Bengio, Y. Convolutional neural networks for mesh-based parcellation of the cerebral cortex. 2018.

Defferrard, M., Bresson, X., and Vandergheynst, P. Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in neural information processing systems*, pp. 3844–3852, 2016.

Gong, S., Chen, L., Bronstein, M., and Zafeiriou, S. Spiralnet++: A fast and highly efficient mesh convolution operator. In *Proceedings of the IEEE International Conference on Computer Vision Workshops*, pp. 0–0, 2019.

Hoogeboom, E., Peters, J. W. T., Cohen, T. S., and Welling, M. HexaConv. In *International Conference on Learning Representations (ICLR)*, 2018.

Kipf, T. N. and Welling, M. Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR*, 2017.

Kondor, R., Lin, Z., and Trivedi, S. Clebsch-gordan nets: a fully fourier space spherical convolutional neural network. In *NIPS*, 2018.

Lai, Y.-K., Jin, M., Xie, X., He, Y., Palacios, J., Zhang, E., Hu, S.-M., and Gu, X. Metric-driven rosy field design and remeshing. *IEEE Transactions on Visualization and Computer Graphics*, 16(1):95–108, 2009.

Maron, H., Galun, M., Aigerman, N., Trope, M., Dym, N., Yumer, E., Kim, V. G., and Lipman, Y. Convolutional neural networks on surfaces via seamless toric covers. *ACM Trans. Graph.*, 36(4):71–1, 2017.

Masci, J., Boscaini, D., Bronstein, M. M., and Vandergheynst, P. Geodesic convolutional neural networks on riemannian manifolds. *ICCVW*, 2015.

Monti, F., Boscaini, D., Masci, J., Rodolà, E., Svoboda, J., and Bronstein, M. M. Geometric deep learning on graphs and manifolds using mixture model cnns. *CoRR*, abs/1611.08402, 2016. URL http://arxiv.org/abs/1611.08402.

Perraudin, N., Defferrard, M., Kacprzak, T., and Sgier, R. Deepsphere: Efficient spherical convolutional neural network with healpix sampling for cosmological applications. *Astronomy and Computing*, 27:130–146, 2019.

Poulenard, A. and Ovsjanikov, M. Multi-directional geodesic neural networks via equivariant convolution. *ACM Transactions on Graphics*, 2018.

Salti, S., Tombari, F., and Di Stefano, L. Shot: Unique signatures of histograms for surface and texture description. *Computer Vision and Image Understanding*, 125: 251–264, 2014.

Schonsheck, S. C., Dong, B., and Lai, R. Parallel Transport Convolution: A New Tool for Convolutional Neural Networks on Manifolds. *arXiv:1805.07857 [cs, math, stat]*, May 2018.

Sosnovik, I., Szmaja, M., and Smeulders, A. Scale-equivariant steerable networks. In *International Conference on Learning Representations (ICLR)*, 2020.

Sun, Z., Rooke, E., Charton, J., He, Y., Lu, J., and Baek, S. Zernet: Convolutional neural networks on arbitrary surfaces via zernike local tangent space estimation. *arXiv preprint arXiv:1812.01082*, 2018.

Thomas, N., Smidt, T., Kearnes, S., Yang, L., Li, L., Kohlhoff, K., and Riley, P. Tensor Field Networks: Rotation- and Translation-Equivariant Neural Networks for 3D Point Clouds. 2018.

Verma, N., Boyer, E., and Verbeek, J. Feastnet: Feature-steered graph convolutions for 3d shape analysis. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2598–2606, 2018.

Weiler, M. and Cesa, G. General E(2)-equivariant steerable CNNs. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2019. URL https://arxiv.org/abs/1911.08251.

Weiler, M., Geiger, M., Welling, M., Boomsma, W., and Cohen, T. 3D Steerable CNNs: Learning Rotationally Equivariant Features in Volumetric Data. In *NeurIPS*, 2018a.

Weiler, M., Hamprecht, F. A., and Storath, M. Learning steerable filters for rotation equivariant CNNs. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018b.

Winkels, M. and Cohen, T. S. 3D G-CNNs for pulmonary nodule detection. In *Conference on Medical Imaging with Deep Learning (MIDL)*, 2018.

Worrall, D. and Welling, M. Deep scale-spaces: Equivariance over scale. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2019.

Worrall, D. E. and Brostow, G. J. Cubenet: Equivariance to 3D rotation and translation. In *European Conference on Computer Vision (ECCV)*, 2018.

Worrall, D. E., Garbin, S. J., Turmukhambetov, D., and Brostow, G. J. Harmonic Networks: Deep Translation and Rotation Equivariance. In *CVPR*, 2017.

Zhao, F., Xia, S., Wu, Z., Duan, D., Wang, L., Lin, W., Gilmore, J. H., Shen, D., and Li, G. Spherical u-net on cortical surfaces: Methods and applications. *CoRR*, abs/1904.00906, 2019. URL http://arxiv.org/abs/1904.00906.

A. Deriving the Kernel Constraint

Given an input type ρ_{in}, corresponding to vector space V_{in} of dimension C_{in} and output type ρ_{out}, corresponding to vector space V_{out} of dimension C_{out}, we have kernels $K_{\text{self}} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}$ and $K_{\text{neigh}} : [0, 2\pi) \to \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}$. Following Cohen et al. (2019b), we can derive a constraint on these kernels such that the convolution is invariant.

First, note that for vertex $p \in M$ and neighbour $q \in \mathcal{N}_p$, the coefficients of a feature vector f_p at p of type ρ transforms under gauge transformation $f_p \mapsto \rho(-g) f_p$. The angle θ_{pq} gauge transforms to $\theta_{pq} - g$.

Next, note that $\hat{f}_q := \rho_{\text{in}}(g_{q \to p}) f_q$ is the input feature at q parallel transported to p. Hence, it transforms as a vector at p. The output of the convolution f'_p is also a feature at p, transforming as $\rho_{\text{out}}(-g) f'_p$.

The convolution then simply becomes:

$$f'_p = K_{\text{self}} f_p + \sum_q K_{\text{neigh}}(\theta_{pq}) \hat{f}_q$$

Gauge transforming the left and right hand side, and substituting the equation in the left hand side, we obtain:

$$\rho_{\text{out}}(-g) f'_p =$$
$$\rho_{\text{out}}(-g) \left(K_{\text{self}} f_p + \sum_q K_{\text{neigh}}(\theta_{pq}) \hat{f}_q \right) =$$
$$K_{\text{self}} \rho_{\text{in}}(-g) f_p + \sum_q K_{\text{neigh}} \theta_{pq} - g) \rho_{\text{in}}(-g) \hat{f}_q$$

Which is true for any features, if $\forall g \in [0, 2\pi), \theta \in [0, 2\pi)$:

$$K_{\text{neigh}}(\theta - g) = \rho_{\text{out}}(-g)\ K_{\text{neigh}}(\theta)\ \rho_{\text{in}}(g), \quad (8)$$
$$K_{\text{self}} = \rho_{\text{out}}(-g)\ K_{\text{self}}\ \rho_{\text{in}}(g). \quad (9)$$

where we used the orthogonality of the representations $\rho(-g) = \rho(g)^{-1}$.

B. Solving the Kernel Constraint

As also derived in (Weiler & Cesa, 2019), we find all angle-parametrized linear maps between C_{in} dimensional feature vector of type ρ_{in} to a C_{out} dimensional feature vector of type ρ_{out}, that is, $K : S^1 \to \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}$, such that the above equivariance constraint holds. We will solve for $K_{\text{neigh}}(\theta)$ and discuss K_{self} afterwards.

The irreducible real representations (irreps) of $\mathrm{SO}(2)$ are the one dimensional trivial representation $\rho_0(g) = 1$ of order zero and $\forall n \in \mathbb{N}$ the two dimensional representations of order n:

$$\rho_n : \mathrm{SO}(2) \to \mathrm{GL}(2, \mathbb{R}) : g \mapsto \begin{pmatrix} \cos ng & -\sin ng \\ \sin ng & \cos ng \end{pmatrix}.$$

Any representation ρ of $\mathrm{SO}(2)$ of D dimensions can be written as a direct sum of irreducible representations

$$\rho \cong \rho_{l_1} \oplus \rho_{l_2} \oplus ...$$
$$\rho(g) = A(\rho_{l_1} \oplus \rho_{l_2} \oplus ...)(g) A^{-1}.$$

where l_i denotes the order of the irrep, $A \in \mathbb{R}^{D \times D}$ is some invertible matrix and the direct sum $\oplus$ is the block diagonal concatenations of the one or two dimensional irreps. Hence, if we solve the kernel constraint for all irrep pairs for the in and out representations, the solution for arbitrary representations, can be constructed. We let the input representation be irrep ρ_n and the output representation be irrep ρ_m. Note that $K(g^{-1}\theta) = (\rho_{\text{reg}}(g)[K])(\theta)$ for the infinite dimensional regular representation of $\mathrm{SO}(2)$, which by the Peter-Weyl theorem is equal to the infinite direct sum $\rho_0 \oplus \rho_1 \oplus ....$

Using the fact that all $\mathrm{SO}(2)$ irreps are orthogonal, and using that we can solve for $\theta = 0$ and from the kernel constraints we can obtain $K(\theta)$, we see that Eq. 8 is equivalent to

$$\hat{\rho}(g) K := (\rho_{\text{reg}} \otimes \rho_n \otimes \rho_m)(g) K = K$$

where $\otimes$ denotes the tensor product, we write $K := K(\theta)$ and filled in $\rho_{\text{out}} = \rho_m,\ \rho_{\text{in}} = \rho_n$. This constraint implies that the space of equivariant kernels is exactly the trivial subrepresentation of $\hat{\rho}$. The representation $\hat{\rho}$ is infinite dimensional, though, and the subspace can not be immediately computed.

For $\mathrm{SO}(2)$, we have that for $n \geq 0$, $\rho_n \otimes \rho_0 = \rho_n$, and for $n, m > 0$, $\rho_n \otimes \rho_m \cong \rho_{n+m} \oplus \rho_{|n-m|}$. Hence, the trivial subrepresentation of $\hat{\rho}$ is a subrepresentation of the finite representation $\tilde{\rho} := (\rho_{n+m} \oplus \rho_{|n-m|}) \otimes \rho_n \otimes \rho_m$, itself a subrepresentation of $\hat{\rho}$.

As $\mathrm{SO}(2)$ is a connected Lie group, any $g \in \mathrm{SO}(2)$ can be written as $g = \exp tX$ for $t \in \mathbb{R}$, $X \in \mathfrak{so}(2)$, the Lie algebra of $\mathrm{SO}(2)$, and $\exp : \mathfrak{so}(2) \to \mathrm{SO}(2)$ the Lie exponential map. We can now find the trivial subrepresentation of $\tilde{\rho}$ looking infinitesimally, finding

$$\tilde{\rho}(\exp tX) K = K$$
$$\iff d\tilde{\rho}(X) K := \frac{\partial}{\partial t} \tilde{\rho}(\exp tX)|_{t=0} K = 0$$

where we denote $d\tilde{\rho}$ the Lie algebra representation corresponding to Lie group representation $\tilde{\rho}$. $\mathrm{SO}(2)$ is one dimensional, so for any single $X \in \mathfrak{so}(2)$, K is an equivariant map from ρ_m to ρ_n, if it is in the null space of matrix $d\tilde{\rho}(X)$. The null space can be easily found using a computer algebra system or numerically, leading to the results in table 1.

C. Additional details on the experiments

C.1. Embedded MNIST

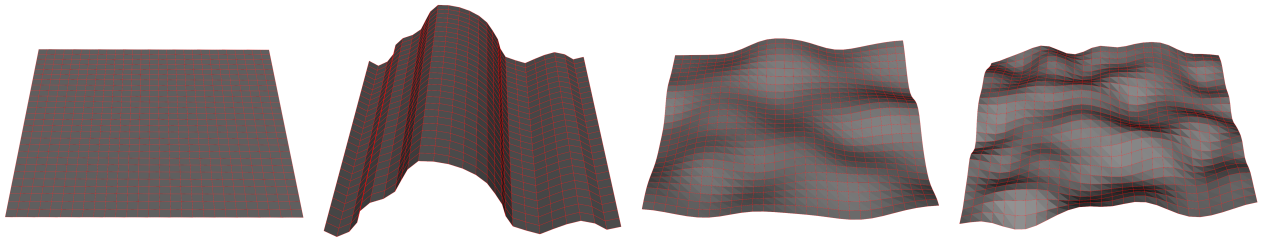
To create the intrinsically curved grids we start off with the flat, rectangular grid, shown in figure 5(a), which is embedded in the XY-plane. An independent displacement for each vertex in Z-direction is drawn from a uniform distribution. A subsequent smoothing step of the normal displacements with a Gaussian kernel of width σ yields geometries with different levels of curvature. Figures 5(c)-5(h) show the results for standard deviations of 2.5, 2, 1.5, 1, 0.75 and 0.5 pixels, which are denoted by their roughness $3 - \sigma$ as 0.5, 1, 1.5, 2, 2.25 and 2.5. In order to facilitate the generalization between different geometries we normalize the resulting average edge lengths.

The same GEM-CNN is used on all geometries. It consists of seven convolution blocks, each of which applies a convolution, followed by a RegularNonlinearity with $N = 7$ orientations, batch normalization and dropout of 0.1. This depth is chosen since GEM-CNNs propagate information only between direct neighbors in each layer, such that the field of view after 7 layers is $2 \times 7 + 1 = 15$ pixel. The input and output types of the network are scalar fields of multiplicity 1 and 64, respectively, which transform under the trivial representation and ensure a gauge invariant prediction. All intermediate layers use feature spaces of types $M\rho_0 \oplus M\rho_1 \oplus M\rho_2 \oplus M\rho_3)$ with $M = 4,\ 8,\ 12,\ 16,\ 24,\ 32$. After a spatial max pooling, a final linear layer maps the 64 resulting features to 10 neurons, on which a softmax function is applied. The model has 163k parameters. A baseline GCN, applying by isotropic kernels, is defined by replacing the irreps ρ_i of orders $i \geq 1$ with trivial irreps ρ_0 and rescaling the width of the model such that the number of parameters is preserved. All models are trained for 20 epochs with a weight decay of 1E-5 and an initial learning rate of 1E-2. The learning rate is automatically decayed by a factor of 2 when the validation loss did not improve for 3 epochs.

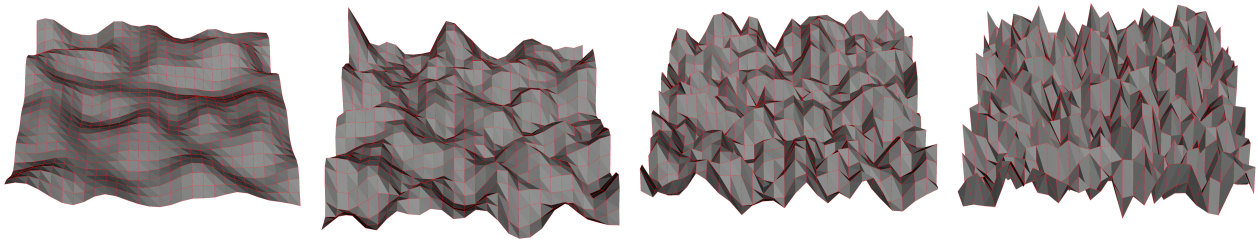
The experiments were run on a single TitanX GPU.

C.2. Shape Correspondence experiment

All experiments were ran on single RTX 2080TI GPUs, requiring 3 seconds / epoch.



(a) Flat embedding (b) Isometric embedding (c) Curved, roughness = 0.5 (d) Curved, roughness = 1



(e) Curved, roughness = 1.5 (f) Curved, roughness = 2 (g) Curved, roughness = 2.25 (h) Curved, roughness = 2.5

Figure 5. Examples of different grid geometries on which the MNIST dataset is evaluated. All grids have 28×28 vertices but are embedded differently in the ambient space. Figure 5(a) shows a flat embedding, corresponding to the usual pixel grid. The grid in Figure 5(b) is isometric to the flat embedding, its internal geometry is indistinguishable from that of the flat embedding. Figures 5(c)-5(h) show curved geometries which are not isometric to the flat grid. They are produced by a random displacement of each vertex in its normal direction, followed by a smoothing of displacements.